

Active InferAnt Stream 013.1 ~ NoOrg is a Real Org

ActiveInferAntStream | NoOrg is a Real Org | 1:44:00

All right. Hello, everyone. Welcome. I'm going to begin this stream with a GitHub push to the Octave Influence Institute GitHub account. No org. So, start this big push, and then we are going to be in the game. All right. First, where we're headed. We are going to be designing cars for butterflies, and we're going to be doing that by orchestrating a ton of different kinds of different agents, LLM and otherwise. It's actually even a lot more general than that. So, stick around, find out, and if you're in the live chat, looking forward to your comments and questions. So, let's pull back a second. This is Active Influence Stream 13.1. It's April 25th, 2025. I'm Daniel Friedman, and the title of this stream is No Org is a Real Org.

Let us confirm that GitHub is still pushing. We'll reload it soon. All right. Big picture.

This is part of some ongoing projects that are related to organizations, documentation, and multi-agent systems. This is open source CC BY NC SA 4.0. Now that the GitHub push has come through, we will reload the repo, which was empty up until now, and we see everything now. So, let us start with looking at the outputs from the example. So, where are we headed here? We're headed into extremely flexible, bottom-up, five-coded, open source, customizable, multi-agent orchestration using an enormous repository of different documents and templates and tools. As just one example, and I will look forward to people suggesting more examples in the chat, and we'll do a bunch of them. Where this is going to go

is in the example number eight. At the top of the script is the convent. So, this was about designing eco-friendly transportation solution for pollinators. To design a lightweight vehicle was the objective. There are some constraints related to toxicity, and the target users are butterflies, actually, but butterfly conservation organizations as well.

So, just to sort of confirm this, I'll wait until somebody makes a suggestion, and then I will modify the project config to be whatever other invented scenario we want. And where that takes us is we get a few different visualizations. So, here's a visualization that I'm going to

to do. First, the research unit does requirements analysis through LLMs. Sends some messages out to the development unit, out to the community engagement unit, budget allocation, all these different rule-based and LLM-based agents that are sending messages to and from each other. And that ends up getting okayed by a board. There's a bunch of different steps. We'll have different examples. And then finally, it results in the marketing department doing some calls to some image

generation services. And then finally, it's a bunch of different types of service. And then finally, we'll have different examples of the product. So, we can kind of pick what type of organizational structure and flow we want. And that is noorg.

So, we will return to how the example works in a little bit more detail. First, I'm going to noorg.mz.

Again, anyone who writes a comment or an idea or a question in the chat, I will try to incorporate this. Okay.

Why noorg? Noorg is not an organization. It's a meta-organization, a way of thinking about how organizations work. It is like cognition is to organization as meta-cognition is to noorg.

And that draws a lot from MediGov, and that's going to come into play later in the stream.

So, part one, noorg is not a real organization. It's not an organization. It is a template and a grammar and system for organizations.

Kind of like how metacognition is about cognition. But at the same time, noorg, if you break it up a little differently,

not that it's not an organization, but noorg, knowledge, as in new, meets RG, which could be renormalization group, it could be role-playing game, and it is a real org.

This isn't just theory. Noorg is a functioning entity, a living system where different kinds of interaction generate tangible outcomes and structure.

The no. The no. Acknowledges the unconventional, the emergent, the resistance, the rigid definition, yet affirms its genuine existence and operational reality.

Or as they say, no organization, no organization. No organization, no organization.

And this is not an organization, yet, and perhaps, it functions just as one.

So, after all, was it a real organization?

People talk a lot about agents and agents in organizations.

However, at the tactical and the operational level, this is coming down to, among other things, documentation, structure, and process flows.

We are at a point where, with a qualitative understanding of this and the above, plus a little bit of i-coding, it is possible to develop some ground-up software packages and paradigms for flexible design, development, and deployment of agents in an open-source fashion

that is agnostic of implementation, structure, function, and process flow.

Knowledge is relational.

The unit paradox.

We will look at the units.

Here is a quick look at the Obsidian documentation graph of what the unit structure looks like.

So, when you dispatch those messages, think about these units and all that they have to do.

Here's operations unit, one of the core units.

And here's all of the documents that are connected to this massive, well-structured, recent theory, units repository.

We're going to get there.

What is a unit?

What is nation?

What is a tool?

What is a finite?

What is an infinite game?

How do we play?

How does the map redraw itself?

How do we diagram out these different kinds of units and functionalities?

How do we explore pattern over prescription and seek in emergent order?

How do we come to understand that knowledge lives in practice and this fundamental paradox of the empty

center, the donut, the florist?

Where and how do duality and recursion come into play when thinking about the sacred geometries of organization of living systems?

How can we use different kinds of interactivity to have this holistic, synesthetic understanding of structure for psychophysical systems?

Where does that take us?

Where does that take us in terms of the Koan-like understanding of structure?

Let us go to the examples after briefly looking at what the units are.

So here is an Obsidian

representation, graphical view, of these different repos.

We will go into the

charter of the finance.

Here we are looking at it in Obsidian, and here it is in

cursor. These are all

files, and there's a different amount in different ones of them, but they all have

some core files like the charter, the index, etc. Let's go to

education. So here's the charter

of education, and it's written out in plain text, and what that

looks like in cursor, in the education charter,

is these double bracketed links. So this

is how Obsidian knows that it is in an Obsidian

knowledge base, and that gets rendered out like this.

So, education charter, index

of different documents within the education unit.

And this can go to an arbitrary level of detail.

So let's just start up a new

chat over here,

drag over education

in the education unit,

comprehensively add sections

and documents

documents

for everything

aid

and education

unit

or

department

rights

So, we'll look at that process.

and the idea with this is they are human readable, and there's ways to render in and out of Obsidian knowledge bases, but human readable forms that give these semantic interfaces that are going to come into play in several ways. So, at one level, just by having these files in the cursor repo, when we then go and code something else, we can see that.

So, at one level, just by having these files in the cursor repo, when we then go and code something else, somewhere else, using prompt engineering, then it's going to find some of this information relevant.

So, that's a very qualitative, semantic, proximity-based approach, like if we were going to go over to some folder and say, write an education paradigm. And then in the education folder, there's already a curriculum, and there's already these different kinds of things, like AI principles curriculum.

So, education could host different information on initiatives, their performance indications, different reading notes that they've had within subgroups or different teams within their unit. So, it's like an organizational structure. And all of that is linked up in the unit directory.

So, here's the big picture of the unit. This is just one way to draw it out. So, high level structure. Here's different organizational relationships. Let's look at it in GitHub. Units. Units are out.

So, in GitHub, the double brackets don't render like they do in Obsidian. It's still there though. Still plain text. Part computing of plain text.

Organizational structure. Almost cool. It's like it's redacted. It's like it doesn't really matter which one they specifically are. But you can have detailed relationships with executive, knowledge, corporate, infra, ops, product tech, revenue.

Different templates and paradigms for strategy, tactics, and operations. Different information flow. Different unit structure. And the way that those link up. And then the dense linking.

In the hypertext. And the hypertext. And in the obsidian form. Tables of information. Interfaces with different communities. Cross functionality. These can get spun out to the empty. And they have all kinds of information that is both at this kind of representation.

Totally readable.

Totally readable, representable, screenshotable by people. Also plain text, versionable. Like you can version it in GitHub. That's what we're going to be doing a lot of. And then also it can be used for machine interpretability. Legibility of documentation. And all of these different kinds of features.

So here is what's. So here is. So here is. So here is. So here is. Updates happening. Updates happening. In the education unit. So it.

Downs building. Downs building. Downs building. Downs building. Downs building.

the reading, made a new structure for the curriculum. Now it's just writing and chugging

along, writing different materials for the education. You can add all kinds of features

there. So this project started as thinking, well, how can we generalize across organizational

structures? How can we look at different units and frame their relationship very generically? Kind of like a Linux kernel. The Linux distribution is not necessarily set up with your username and your password and your wallpaper. It might be some defaults, or it might just be a placeholder, but then this can be rendered or deployed, kind of like, again, a Linux kernel for organizations. And it's an opportunity to flesh out some really exciting and interesting things in this generic platform. So here's another example of that. Within the security department, we have cognitive security, digital security, and physical security. Of course, cognitive being the most general, of all. And there's only a few documents in here now. But we can start to play around in the security unit. Oh, wait a minute. No playing in the security unit. And start to explore, what does that generic framework for cognitive security in an organization look like? And it's like, as always, it's the generic and the specific, right? Genus and species. Generalizations and menu particulars. So we might draw and abduct or induct or deduct in different ways, thinking about, like, this would be super cool for a research unit in general. What would be their focus areas in general?

Or there could be a sub-research area, like, it could be the taste research unit.

So these kind of nested and interacting units give a lot of opportunity to take the generalization right up to, and probably a little bit beyond, the point of being generic, and end up with all of these different kinds of sub and fractal and nested interacting structures.

So we will add an education subunit for language learning. Be comprehensive.

Be comprehensively.

Okay, so that's kind of the unit structure. And I believe that that is some of the utility and novelty that you see in this situation, which is that it's an Obsidian open source, plaintext approach to representing organizational structures in the generic and in the specific case, using Obsidian as a way to visualize this no org, right? Because it's not an organization.

So all these different connections, and people can look at how different things connect up.

There's the unit directory. All the things that it's connected and draws.

Okay, so that is the unit component of this package.

Push the GitHub update.

Okay.

Okay, back to the screen agenda, which I'll just break down as we get through things.

So, the unit directory and the markdown relationships was what I began with.

Was building this out and just continuing to say, okay, now add a new unit.

Now add this unit.

Now link these two units.

Now add this unit.

Now add more within this unit.

Just what other units would be relevant?

Just keep going and going at all those different meta organizational, meta governance type structures.

Let's add a department of meta go.

Let's...

In...

In...

In...

In....

You'll slide.

Let's in units.

Make a new unit with all needed documents in.

Form and a governance.

Let it go.

About fractal and scale 3.

And interacting governance processes within and among the units.

Language learning.

Subunit.

We have a language learning folder.

It's been one of the most effective augmenters of live coding.

First off, the multi-shot agent mode in-person.

And then improved ability to have situational awareness in the file structure.

And being able to have new folders get made.

And make files iteratively.

All that kind of stuff.

Plug it on.

Language learning.

Education unit.

Navigate is in process.

The units folder.

It's again, it's pretty simple in concept.

And it can get rolled out to basically the nth degree of sophistication.

You can get as many generic or specific policies as you want.

Just keep going.

You can continue to work off of the org chart.

Meta org chart slash module structure that is currently in the no org repo.

Or, I think you've probably seen enough, even at this point, just to go into cursor or whatever.

And just say, okay, build me a modular meta organizational structure.

Roll out every single possible thing that could be relevant for xyc.

That'll kind of build you what you want at the right level of detail.

And, you know, tuning and tweaking along the way.

Back to the...

Again, third time to return to this.

But I started with just the obsidian repo.

There was no code in this repo at all.

It was just about, well, if you consider what to consider code.

What you consider a code.

Only in the units repo.

Just exploring.

And I thought, okay, well, wouldn't it be pretty cool to be able to start to deploy custom configurations of agents within those units?

And that is where it went.

So there's a few different other folders to look at.

Agents has a lot of information.

All of these documents.

Ridiculous numbers of documents.

So here, this obsidian window is only looking at the units.

Now this one is pulled out to the broader.org.

So they're both kind of mutating as the MetaGov files come into play.

So here's the broader organization.

Slightly different, much larger network.

A lot of more standalone files because outside of the units directory, there's less obsidian linking.

Here in the agents, again, top level folder, there's all these documents.

And ranging from the, some of these are obsidian linked.

They're more heterogeneous.

Let's look at it in, let's see.

So this is like why putting stuff online makes a semantic impact, even if it's hard to get attention of people in the moment.

It's like all the work that MetaGov and people using the term have done, some of it gets slurped up, uses training data, and that supports later through this kind of airlock of not really citing them.

Sometimes a specific citation or URL or person might pop up, but it's using the semantics of how MetaGovernance was discussed by the people who discussed it.

That's how symbols and words work.

So here's a charter for a MetaGovernance unit.

And then if people want to configure their own organization, as opposed to this non-existent organization that we keep on talking about, without this or with it, you can have knowledge management, you can have a legal department or not.

But it's configurable.

That's kind of cool.

Back to looking at the agents in...

Let's see.

Let's see.

Let's see.

Let's see.

Let's see.

Let's see.

Let's see.

There's some code in the agents folder, but not that much or that.

Here are a large number of agentic semantic coders.

So this is like at the meta-agent implementation level, all these different kinds of schema, just at the conceptual level.

Like if we want to talk about how do we tune performance in multi-agent systems.

Just some pseudo-code, some different ideas, some different patterns.

How do we think about autonomous learning?

How do we think about learning architectures, learning systems, different ways that can be pseudo-code, but this is not meant to be executable directly.

But again, where's all this going with the five code?

Natural language is code.

Code along.

All of that.

And so these kinds of plaintext, which are now happily publicly available in this open source repo, it plays a role.

So if someone says, change tracking and make new changes yellow, that's going to move the needle in some little way into who knows how long later.

Someone says, make a visualization for change tracking.

And that yellow is going to put its little syntactic thing on the scale.

So that's what's happening in the system, agent system folder.

So what's happening in the system is different kinds of looser semantics using natural language, but again, heavily human interpretable and heavily machine readable for all these kinds of qualitative interpretable elements of agents.

So there are guides, just a few of them, but they are guides to different situations where you might want to do the pronunciation work.

So here's like a swarm implementation.

Some of this refers to some real code.

Some of it doesn't.

But the whole point and what we see is you just say, okay, build it out where it's missing.

Use this pattern and go from there.

And that will basically get you ready to go.

So guides, examples.

Let's look at it in both places.

Here are, again, markdown files of different examples of multi-agent behavior.

So part of it is like, why state the objects?

But it gives up, if you want to just say, go to the continent or go to this region or go to this city block or go to this nanometer.

There's multiple levels of semantic precision.

And it's not that more is simply better.

So actually having a really elegant, small, concise way to be like, when we talk about task execution behavior, here's broadly what we're talking about.

We mean that this is what happens.

Or when we talk about learning, this is what we do.

Those are helpful.

And then more technical details can always be scaffolded and layered in.

Maybe there's programming languages, coding agents, all this kind of stuff we don't even know about.

I would highly believe that.

So why don't I work at a level, like a meta organizational level or a meta programming language level or a meta linguistic level that can always be deployed in different ways?

What do we mean by coordination behavior?

In the communication folder?

Just a few documents about communication specifically.

It's like, you can keep on growing these documents.

And then it's, okay, now let's have a sub folder for just monitoring.

Let's have a sub folder for just task execution.

Keep on growing out and monitoring and improving these documents and modifying them as they are.

And architectures, which give a few documents that are related to architectural concepts in agent design, like cognitive models.

So the next time that there's a new paper or a new social media post or some other kind of cognitive model, it can just get plugged right in here.

So it's kind of like a new restaurant opening in a region.

It can already be linked in to what is the region, what is the kind of food, it's a restaurant, all these background pieces of information that already bring it into this very interoperable system.

So let's say agent design.

Archive has a few different prior tests.

Let's see if any of them are relevant.

This one was a multi-agent sorting algorithm.

Just using this swarm dispatch paradigm to do some sorting on different numbers.

This one was a test suite that was written to do a lot of the functional testing for multi-agent systems.

So looking at task performances.

And then it got up to doing the towers of Hanoi.

So this is like a team of agents that were spun up and spun down that were playing these towers of Hanoi.

So people see more.

That's in the archive.

Alright.

Then there's dist.

So this was another approach to writing out different kinds of multi-agent systems.

Different ways.

There's multiple different frameworks.

Obviously, that would be the same least that there's many different ways to do multi-agent modeling.

So these are just showing that there's a few different ways to have, in this case, JavaScript agents that have different kinds of capacities, different kinds of abilities.

That could be augmented or the semantics could be transferred to all different kinds of settings.

That's the dist.

And this is a message passing architecture that actually does have some function.

But it's not the main one.

And last, before we get to the examples of this TypeScript code, is the docs folder.

The metadocs.

The paradocs.

The docs of docs.

Here is a vast number of folders of documents.

That some of them have a ton of information.

Some of them have just a little bit of information.

And they are just another way to include semantics related to these topics.

Pretty interesting to think about like where and how is the folder structure, just the regular computer nested folder structure, sufficient for conveying the kinds of structural and infra structural and super structural information that we want to convey.

And where do we want to use like graphical databases, other methods.

So we won't look through all these documents of course, but they're all up there.

And they come into play when you use cursor.

Scan the whole prequel carefully.

Then continue to add link and relevant metagov stuff.

So docs folder has all different kinds of relevant documentation that can be brought into play.

In different ways.

So I can again, be used at that qualitative level.

Of just like, here is AGI safety.

Well then, if you type in something related to that, or something happens related to that, then whatever is in this file is going to come into more play than it would have otherwise.

There is a bunch of sections.

Let's see if there's any other fun ones.

Core documentation.

That's the documentation on the documentation on the documentation.

Different ways to do automation.

The Git is actually a real one.

FractiGit is a fractal Git program.

Pro.

So, just tucking away, it's kind of like you're looking through a library, a hyper library of documentation.

And sometimes there's a little bookmark, sometimes there's a little pheromone trail, other times it's like, huh, that's a whole functional piece of software.

So in this case, the fractal Git system, as inspired by Pro, of course, is a fractal Git coordinator that spins up and down and makes different kinds of Git operations on a fractal Git structure.

So that kind of extends the Git concept to these nested, quantum, superpositional repos.

But it could be anything, but it actually does have some utility.

So, being able to make repos, spin up, spin down, clone, merge, all those different kinds of operations on fractal Git operations, give a lot of flexibility because, again, though it's probably or possibly not the most high bandwidth or high utility representation structure to use the computer nested system.

And even though it might not be the exact set of commands that are the most useful in all possible universes, like the default Linux ones, there's a ridiculous amount of training data on these kinds of things.

So, using folder structures and Git, plain text, you get really far and do a lot.

So, MediGov still chugging away.

See if there's any other types of documentation that are fun.

Different integrations with messaging platforms or could get integrated to Mattermost, Matterbridge, be able to bridge across different messaging platforms.

There's some related sections to the units, a lot of maintenance information.

So, what at this meta-documentational level, what are good documentation maintenance practices?

That can be specified whether you're talking about documentation for building a car, driving a car, whatever it is.

This is the meta on that.

And then, today or tomorrow, you can send a person, undergrad, code agent, whatever, and say, go to the maintenance folder and do da-da-da-da-da-da.

And all of that, screenshot management.

We've been in some situations where the proof of screenshot was make or break.

It was sink or swim.

So, having the screenshot management on point, it's like, I've never seen it written out that way.

But I could see why it matters once it was written out a way.

Meta documentation in the meta documentation is actually quite a large section with information on the information on structuring information.

Helping provide some meta documentation on this docs folder.

Different metrics.

So then, when a prompt is used, like, make it secure.

Or make it high quality.

Or make it performance.

Those can be just filled up with the, make it successful.

Make it valuable.

Those are going to be just drawing on the base semantic distribution of whatever model you use.

Plus, whatever indirect context you have in the code base, or scope.

Plus, whatever are in these documents.

So, if you super emphasize that quality means elegance, then that is going to move the needle in that direction.

Let's just see if there's any other.

Then we'll go to the examples.

Use your interface.

Troubleshooting.

Training.

Just open these up.

See what's there.

And then we will close Obsidian and work more on the code examples.

Get back to that butterfly car.

Meanwhile, if anyone watching can have a suggestion for a, a, something that they want invented through this multi-agent orchestration or new setting, then check.

Yeah, prompts.

This is just kind of, there's a lot of ways to do prompt management, of course.

But, here, the prompt folder, so then when you write a prompt, or even not, do good knowledge management.

And as usual, mermaid diagrams coming in super clutch.

Very few tokens.

Very easy to make a difference on a text model.

And, it renders plain text, etc. for, for high information throughput for, for people and for visual accessibility.

Props folder.

Quality assurance.

Research.

Search.

Security.

Different security concepts.

Different ways to, to discuss it.

Setting up.

Software.

Standards.

Strategies.

Systems.

Systems.

Systems science.

More systems.

Templates.

I mean, I just, it, it was kind of like, just to continue to play out.

What does it look like when you keep on nesting templates?

Like, okay, well we could have templates about, media notes, that you can get into more and more levels of detail with templates on, on just more and more and more.

Or not, just use the base, probabilistic distribution of the language model.

But, if the template is in place, you can investigate it and it will be done.

Testing.

Tools.

Training.

Tools.

Tools.

Tools.

Tools.

Alright.

That will be it for the Obsidian part.

And, we will look now at the, at where we got here.

Okay.

So, I'm going to focus on the experiment number eights and nine.

Eight is going to be a, seven, eight, nine are some of the more sophisticated ones.

Seven is, uh, and all this goes into the output folder.

Oh, all the, the source code for all of these examples is in SRC.

So, here's where there's a ton of information, utilities, uh, there's a documentation folder inside of SRC, of course.

Core methods, different monitoring, messaging, different patterns, different type scripts, uh, and some agent types.

Which can be up to your concern.

So, we'll probably not go too deep into SRC, but let's look at the examples now.

Uh, so in the multi-unit LLM flow, seven, which we'll just kind of pass through because, uh, eight is a little bit bigger.

So, in this case, put this up in the, pardon me.

So, here's, this was an orchestrated flow.

These things are going to happen in this order.

And this is the summary of, of what happened.

So, the, the, the initial prompt that was given to the, to the innovation unit was, uh, a machine for counting and the heat.

And then this cost, it looks like 67 cents.

Not, not sure if that was accurate.

Probably less than that.

Um, and that summarizes the entire process.

Uh, it is first sent from the innovation unit to assess the idea.

Then, the, the, the, uh, generation of ideas.

And here's the idea generation response.

We'll, we'll look at this in the future.

So, prompt is a fixed prompt.

And then, it gets back.

Okay, here's Antscope.

Ant monitor.

Here's different ideas.

Ant site, Ant secure, Ant connect.

That gets sent, the idea generation gets sent to the market research, security, assessment, and then finally to the board of review.

So, let's look at the board review.

The Ant School project aligns well with our organization's strategic focus on using technology to address environmental concerns, etc.

That is then refined with feedback at the innovation unit.

There's a technical and a brand planning.

And then finally, the final project plan is the now.

So, all of this was just automatically orchestrated based upon this fixed organizational technology.

So, final project plan.

So, it's an executive summary.

With all of the information from the different sections.

So that's 7.

Example 7.

The multi-unit flow.

Okay, 8 is the one that brings in a few other features.

Alright.

You see cursor?

Okay.

Okay.

Uh.

So as shown a little bit earlier, what this script is going to do is demonstrate combining multiple agent types in a single workflow.

There are some LLM calls, there are some rule-based systems, and there's also image generation.

So let's modify this section.

First, anyone watching live, write your idea for a project or the context for a project.

And meanwhile, we'll just look at the butterfly car design to look at where we're going to get.

Which is, all the intermediates are safe.

So here's like the architecture designer.

So the system prompt is your systems architect, system generic prompt.

And then the user prompt is design a lightweight aerodynamic vehicle optimized for butterfly transportation and sink.

And then the output from the architecture designer is this is what, this is the concept.

It's in adjacent.

So it's wide.

So it's not a good idea.

But it deals with, you know, having GDPR for butterflies using different kinds of access control and encryption for butterflies.

So the images folder are where the final prompts are created and sent out.

So here's like the butterfly car, here's a logo, and here's a systems representation.

So let's make another concept just to confirm that it's due to the first patch.

Using control K just to modify it in line.

This is also quite long file.

Over 3000 miles of yard.

Alright.

So let's update this.

So it is designing a physical crypto currency hardware wallet that must be invisible and waterproof for children.

So now it's called Stealth Kids Crypto War.

And all I prompted here using cursor was prompting to change these project objects.

So it could be a different domain, could be a different user group, different constraints, you know, it has to cost zero dollars or whatever it is.

Then over on the right hand side, I'm going to just ask cursor to run 8.

So it will generate up the codes running.

Alright.

So meanwhile, while 8 is running over there, I'm going to look at the example 9.

here's where the output is going to get to.

Let's go to the output.

So this is the exact example 9.

Here's where the output is going to get to.

So this is the executive summary.

This report presents the results of an active inference partially observable Markov decision process model used to analyze temperature changes throughout a narrative.

model was designed to attract temperature information from the story and determine optimal actions based on the inferred state of the environment.

So this has information on this.

Let's look at it here.

Report.

So here's beliefs.

Belief updating about temperature.

But the way that this one works is that a story is defined in natural language in the product.

And then first, an LLM is going to be called to say, what do you think the temperature was at each beginning, middle, and end of the story?

And then the outputs of that, structured outputs from that LLM, end up getting taken as observations for a PalmVP that then runs an active inference algorithm.

So this is more of just a two step program.

We've got eight running in the background.

So this one, let's also do a little modification.

So here in this story, it was crisp in the morning.

Then the sun was high.

So it's kind of indirectly saying it was, it was, the weather was perfect for hiking.

Neither too hot nor too cold.

But in the morning, it was quite cold.

Outside, the temperature continued to drop.

So I kind of went from like medium to cold, from cold to medium to very cold.

[illegible]

Let's here.

Just run.

Nine.

There's some document chunking and splitting techniques.

So here we are looking at output.

Oh, did we save it?

Okay.

Oh, it's still working in the butterfly situation.

This is another freshly...

A different butterfly.

Wow.

It's like a spaceship for butterflies.

Just like it should be.

And we'll see the third one just appear.

Butterfly.

Right?

It's just because I didn't save it.

Not saved.

Great.

We just changed the prompt.

And eight.

And eight again.

Same button.

That will show up in the folder as the cellpid's crypto wallet.

Okay.

While that's going.

All right.

We did the GitHub push.

We looked at the outputs from the examples.

We looked at the butterfly car design.

And now we're making the cellpid's crypto wallet.

We looked over the sections of no org.

This is not an organization.

No org is it.

Reviewed the unit.

Meta architecture.

Talked about the documentation.

All right.

There's two more sections.

That.

A few more sections.

First.

I'm going to look at cognicism.

By my colleague John Ash.

And for what happens when we input the concepts of cognicism into this framework.

Then we will go to Medica.

And pick one of their projects.

Possibly the constitutions.

Possibly a living manifesto for online spaces.

And see where that goes in this no org setting.

And then look at any comments in the live chat.

And like if somebody wants to suggest another cool product to design.

Or some other funny thing.

So here's.

Now here's new active inference output.

So let's look at this report.

Okay.

Read this report.

Similar.

But.

I think people get it.

This is a very.

Useful pattern though.

What's happening in 9.

Which is.

Taking in.

Natural language.

And then having MLM.

Do some kind of inference on it.

Like.

Tell me how many characters are in the story.

Or how many facts support this.

Versus how many facts support that.

Or what temperature do you think it is.

Or how much do you think that would have cost.

All those kinds of questions.

Not that it.

Necessarily returns.

An accurate understanding.

But.

First off.

You can sample it a bunch of times.

At different temperatures.

And that enables you to get.

A really good inference.

On the latent space.

Of the LLM.

So you can say.

What GPS coordinate.

In this situation, the orchestration pattern is fixed within the example, but you can imagine creating a situation where it's dynamic. So let's just try that.

say,
even the functionalities
of these two. Let's just focus on just the LLM orchestration.
of this example. Write a new example tab where there is a content block at the top so that the user inputs
natural language for the
organization.
for what kind of organization flow they want to see. We'll see if we can one-shot this. This might be a few
steps.
and first an LLM call assesses dot and returns whatever structure format you need so that that organizational
flow can be
implemented using the approach. Similarly, with all
intermediates saved and the comprehensive org chart HTML visualization. With the org chart as
as compared after that special first step, then take in a prompt from the second part of the
the org chart.
and then take in a prompt from the
form page block. Another variable.
for what that custom org
should get.
okay.
let's look at cognicism and navigation.
let's look back
okay.
due to the size of the margin, it will be left as an exercise for the reader to learn more about what
John Ash is up to what cognicism is about. Medium is a good place to see his work. I'm just going to see if
copying this one glossary document is enough to make some kind of cool example.
so making a new folder.
so making a new folder.
cognizantism.glossary.md
let's see if there's any other blog post that might be useful.
let's just try it with just the last.
okay.
okay.
so that's 10.
in this folder.
templateing off of the
functional
8
write

a
creative
relevant
useful
implementation
of the
cognicism concept
make
it
very relevant
and
flexible
and with
focus
visualizations
and animations
start with the
.cs
background
if you want
to make a
Python
file in the folder
for
analysis
analysis
and visualization
and animation
please do
them.
okay.
alright. so here comes
10.
for the dynamic
board generator.
and then here's the third
of the three possible chats
that are in Christian.

let's look at what we could do

for

Medigo.

okay.

i don't see where the actual document is.

okay.

constitutions of web 3 might be a good one.

so this is work by Seth Bray and Josh Kingham.

and i know they have a good house.

all right. so here they curated some constitutions from different DAOs and different groups.

so just as an example.

if somebody wants to pick a

DAO

let's do

let's do

lexDAO

and say we'll have a good legal constitution.

alright.

i'm gonna do

something pretty similar.

examples

new folder

lexDAO

lexDAO

defile

lexDAO

on state station.

alright.

now

i'm gonna use the same

prompt

with prompt.

templating off of the 8

write

a

ail

ail
!
ail
!

ail ail ail

!

ail ail

!

ail

!

ail

ail

Now, Lex down.

Alright, we are.

Cognizant plug no head.

Lex down here.

Alright, in terms of tool use.

Cursor 49.6.

Mainly using Claw 3.7 Sonnet.

I am also using Gemini 2.5 a little bit.

[illegible]

no specific background

thank you

the meetings are going to be at 16 etc on tuesday and 19 etc on wednesdays

and it's going to be first a design phase of one month

to spec out like what kind of a multi-agent system are you interested to be developing

uh what kinds of agents what kinds of information environments what kinds of flows

all those types of questions and then in june we're going to move it into an implementation phase

so as kind of shown here

it's easy to get a sloppy design and implementation

with basically just kind of natural language prompting or just using .org

um but i think over the coming month

we're going to have a lot of really exciting ways to work at that specification level

and then have strong ways to go from that specification to a high reliability

very well implemented system

so i would uh encourage anyone who's kind of peaked by this stream

to get involved that way or if it's after then just probably still all good

yep getting a lot of errors with me calling the edit file but at least sometimes it's getting through

it's just having to do it in shorter sections

um one more piece that doesn't require the LLM

this is a paper that i came across

notation as a tool of thought

by Kenneth Iverson from IBM research

so a few quotes here

by relieving the brain of all unnecessary work

a good notation sets free

to concentrate on more advanced problems

and in effect increases the mental power of the race

a n blanking

and the quantity of meaning compressed into a small space by algebraic signs

is another circumstance that facilitates the reasonings we are accustomed to carry on by that aid

how though it is

and this uh paper

goes into some pretty exciting directions discussing notation

and about ways to have information

compression and operations through notation

so kind of like Gerl-Euscher-Bach

a notation scheme is developed

like loop and gloop

and it's used to carry out a few different kinds of things like
imagine maybe maybe like this
but now we can make these notation systems
that are just
able to blur the line between the symbolic and the probabilistic
and uh some of these things would take a long time to understand manually
but it would be possible to put this into an LLM
and uh develop implementations
based upon that
like this would be literally like learning a whole new language
but
we could just
put it into a file and say
okay given that this is the language that we speak
let's just see what happens
notation
this is going to be called
tool
uh
thoughts
notation
okay
alright
run the
dynamic org generator
and fix it
if anything
okay
okay
okay
okay
okay
okay
okay
okay
let's see

Okay.

Progressisms in place.

Lex thou in place.

Let's see if the dynamic generator works for 10.

So, of course, this is all happening in real time.

Unscripted, neither I nor the machine.

So, it's more of an experiment to see.

Will it work?

Okay.

Click the open box in notation.

Here, I'll use the control K to use it in line.

Keeping all the information exactly.

Structure this perfect.

And we could pick a different model.

We could use O4.

O4 Mini is also pretty useful.

It's awesome, though.

It's just a little bit...

slow.

In this document, write up a bunch of, well, annotated examples of using this notation.

Guys, on using it.

Any difference?

All right.

All right.

But once it gets started, then it starts to fix it up.

Okay.

Okay.

Yeah.

Several errors in that first draft.

But now it is going to at least continue to go.

Okay.

Okay.

I don't know why it would be a diss.

Yep.

All the L1s are a little slow.

Right.

They are getting the job done.

Let's try something that might be a little devilry.

Okay.

It takes a few sometimes.

But yeah, it could be faster, but at higher MLM costs.

Okay, here we go.

So, Gemini coming through.

Alright, so, structured.

Wow.

Okay.

Using the Control-K in line.

Alright.

Okay, something happened.

Okay, how do you use...

So, more examples.

Just to respect and weave back with the past.

These are all, like, kind of elaborate notation for thinking approaches.

All the stuff.

GNN, AOS, Cerebrum.

All that kind of stuff is like...

Using function with notation in these doctrinal ways.

So, agreement engine.

A lot of similarities.

More of a focus on agreements.

Looks like it hasn't been updated in a couple of years.

But...

Similar concepts.

Different kinds of organizational flows.

And...

Could be...

Could be revisited in R5 era.

Community rules.

Could be revisited in R5 era.

Community rules.
Similarly.
Could be...
Uh...
Community rules.
Similarly.
Could be, uh...
Brought in play.
Missed in R5 era....

Missed in R5....
Missed in R5....

Missed in R5....
Missed in R5....
Missed in R5....

Okay.
Culling the system is good yet.
This file is

Left Down

Okay.

I will take a sip of water, see if anyone writes any questions or ideas in chat, and hope that these finish.

[illegible]

Okay, we've gone faster, let's just do the brawl.

Long, eight, and the same.

Unnecessarily, but I will have to do partial function version stuff.

...budget bonds here.

Maybe it'll end up there, actually.

Okay.

It's the epistemic slot machine.

One more purpose.

So close.

Push an update.

I'll look at the chat.

Update.

So, LS, let's see, all add it to the list.

Okay.

Okay.

Alright, maybe we will get there.

That's always part of the company.

Okay.

Okay.

Ignore and continue.

Okay.

Okay.

Okay.

Okay.

Alright, maybe we will get there.

Okay.

Okay.

Alright, maybe we will get there.

That's always part of the company.

Okay.

Okay.

That's always part of the company.

Okay.

Ignore and continue.

Okay.

Okay.

That's...

Something is happening there.

Okay.

Okay.

That's...

Something is happening there.

Okay.

And here.

Okay.

Okay.

Okay.

Something is happening here.

Okay.

Alright.

We could have gone.

Alright. Awesome.

Alright.

It's incredible.

Actually incredible.

Alright.

Let's do a different organizational topology just to confirm the value.

Let's now run it.

Here we go.

So dynamic org generator.

Alright.

Alright.

So that was...

Okay.

Organization description, organization task.

We are updating this to be an organization that is literally shaped like a skyscraper.

It is like a skyscraper.

Shaped like a mollypop.

With...

A bunch of vertically nested layers.

And on the top layer it is hub and spoke with six connections.

The task for the organization is actually appropriately to produce lollypox.

Okay.

I'll modify the font tree.

Alright.

We'll see if it actually got there.

Okay.

So...

Alright.

We'll see if it actually got there.

So...

I'll modify the font tree.

I'll modify the font tree.

I'll modify the font tree.

I'll modify the font tree.

I'll modify it again.

Alright.

We'll see if it actually got there.

Okay.

Okay.

LexDow is working.

Never give up.

MissDow is working.

MissDow is working.

Okay.

Proposal visualizer.

So, it is hanging here.

Fix that and run it again.

View!

And those changes.

Please.

Then run it again.

Alright. Let's look at the second dynamic org.

See if we got the Lollipop Co.

Who represents the Lollipop Guild?

Of the Lollipop,

War the Lollipop.

They succeeded.

Fantastic.

Okay.

I wouldn't say that's exactly it.

pretty awesome nonetheless.

Let's just see if we can get the home tech going.

New chat.

New chat.

In the home tech.

In the home tech.

Hey there.

In the home tech.

Hey there.

states....

states....

use and elaborate on that so it is a new example comprehensively representing
form tech from M organizational ergonomics positional okay let's now improve it please
summarize exactly what
see we do

...

...

...

...

...

...

...

...

...

...

!

...

...

...

...

...

...

parents let's see if we can get it

okay in closing let's do

in talks dialogues

we'll do who's not first

four dollars pay through in the chat four dollars api cost for all of these examples that i'm

running so like 90 dozens including a lot of the image generation which is probably takes up a large

amount here write a synoptic panoptic all good all beautiful

all truthful

truly hilarious who's on first style vast dialogue regarding all the information in

examples

is

No pressing.

Okay.

I'm going to call it there.

But as soon as the who's on burst comes through.

Alright, see that.

We'll just push the last update.

Finishing the stream.

And then we'll look at the dialogue.

Docks.

Dialogues.

Okay.

There it is.

Alright.

Hope this was useful for some.

It's definitely been fun to work on.

And I'm appreciative you've watched this far.

And attended to.

It's pretty fun.

So looking forward to people's comments and contacting me.

And however.

Alright.

Until next time.

Alright.

Until next time.

Bye.